

Absolutely. **Encapsulation** is one of the four major OOP concepts, but in Python it is often misunderstood as simply “making variables private.” That’s only a small part of it.

# Encapsulation in Python OOP

## 1. What is Encapsulation?

**Encapsulation is the practice of keeping an object’s data and the operations that work on that data together, while controlling how that data can be accessed or modified from outside the object.**

In simple words:

**Encapsulation = Bundle data + methods together and control access to the internal data.**

For example, imagine a bank account.

A bank account has:

Data:

- balance
- account\_number
- owner

Operations:

- deposit()
- withdraw()
- check\_balance()

Instead of allowing everyone to directly manipulate balance, we can make the object responsible for changing its own balance.

**class** BankAccount:

```
def __init__(self, balance):
    self._balance = balance

def deposit(self, amount):
    if amount > 0:
        self._balance += amount
```

```
def withdraw(self, amount):  
    if 0 < amount <= self._balance:  
        self._balance -= amount  
  
def get_balance(self):  
    return self._balance
```

Now:

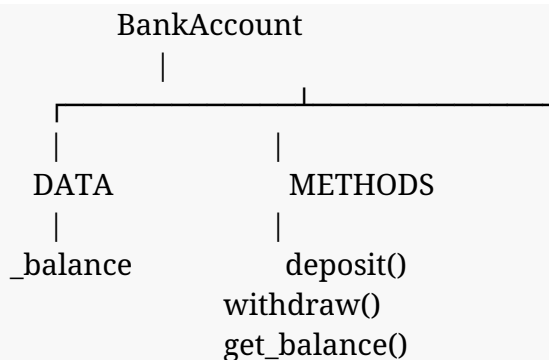
```
account = BankAccount(10000)
```

```
account.deposit(500)  
account.withdraw(200)
```

```
print(account.get_balance())
```

The important idea isn't just `_balance`.

The important idea is:



The object controls how its data changes.

---

## 2. Why Do We Need Encapsulation?

Without encapsulation:

```
class BankAccount:
```

```
    def __init__(self, balance):  
        self.balance = balance
```

Someone can do:

```
account = BankAccount(10000)
```

```
account.balance = -500000
```

Now the object contains an invalid state.

```
balance = -500000
```

That's usually not something we want a bank account to allow.

With encapsulation, we can put rules around modification:

```
class BankAccount:
```

```
    def __init__(self, balance):  
        self._balance = balance
```

```
    def withdraw(self, amount):  
        if amount <= 0:  
            print("Invalid amount")  
        elif amount > self._balance:  
            print("Insufficient balance")  
        else:  
            self._balance -= amount
```

Now the class controls the operation.

---

## 3. Encapsulation Has Two Main Ideas

When learning Python, separate these two concepts:

### 1. Bundling

Put related data and behavior inside a class.

```
class Student:
```

```
    name = "Abhi"  
    age = 20
```

```
    def study(self):  
        print("Studying")
```

Here:

```
Student
├── name
├── age
└── study()
```

Data and behavior are bundled together.

## 2. Access control

Control how internal data should be accessed or modified.

For example:

```
class BankAccount:

    def __init__(self):
        self._balance = 10000

    def withdraw(self, amount):
        ...
```

The `_balance` is intended to be an internal implementation detail.

---

## 4. Encapsulation ≠ Just Private Variables

This is probably the **most important point**.

Many tutorials say:

Encapsulation means making variables private.

That's incomplete.

For example:

```
class Student:

    def __init__(self):
        self.__marks = 90
```

Yes, this demonstrates **data hiding**, but encapsulation is broader.

Encapsulation involves:

Encapsulation

- |
- ├─ Bundling data + methods
- |
- ├─ Controlling access
- |
- ├─ Protecting object state
- |
- └─ Providing controlled interfaces

So:

**Data hiding is a technique used to achieve encapsulation; it is not the entire definition of encapsulation.**

---

## 5. Access Levels in Python

Python doesn't have Java-style strict:

```
public  
private  
protected
```

access modifiers.

Instead, Python mainly uses **naming conventions and name mangling**.

There are three common forms:

```
public  
_protected  
__private
```

---

## 6. Public Members

A normal name is considered **public**.

```
class Student:
```

```
def __init__(self):  
    self.name = "Abhi"
```

```
s = Student()
```

```
print(s.name)
```

Output:

```
Abhi
```

Anyone can access it:

```
s.name  
s.name = "Rahul"
```

So:

```
self.name
```

is public.

---

## 7. Protected Members — `_name`

A single underscore:

```
self._balance
```

means:

“This is intended for internal/subclass use. Don’t access it directly unless you know what you’re doing.”

Example:

```
class BankAccount:
```

```
    def __init__(self):  
        self._balance = 10000
```

You technically **can** do:

```
account._balance
```

Python won’t stop you.

This is important:

`_balance` is **not truly private**.

It's a convention.

---

## 8. Private Members — `_name`

Two underscores trigger **name mangling**.

```
class BankAccount:
```

```
    def __init__(self):  
        self._balance = 10000
```

Now:

```
account = BankAccount()
```

```
print(account._balance)
```

will produce an error similar to:

```
AttributeError
```

Python internally transforms the name approximately into:

```
__balance  
  ↓  
_BankAccount__balance
```

So:

```
account._balance
```

doesn't directly find the attribute.

But:

```
account._BankAccount__balance
```

can technically access it.

Therefore:

Python's `__private` is **not absolute security**. It is primarily **name mangling** that prevents accidental access/name collisions.

---

## 9. Why Does Python Use Name Mangling?

Consider inheritance.

```
class Parent:
```

```
    def __init__(self):  
        self.__value = 10
```

Python internally treats this roughly as:

```
_Parent__value
```

Now a child class can have:

```
class Child(Parent):
```

```
    def __init__(self):  
        self.__value = 20
```

which becomes roughly:

```
_Child__value
```

So they don't accidentally overwrite each other's private names.

Parent object data

↓

`_Parent__value` → 10

Child object data

↓

`_Child__value` → 20

This is one major purpose of name mangling.

---

## 10. Controlled Access

A very common encapsulation pattern is:

Private data

↓

Getter

↓

Read data

Private data

↑

Setter

↑

Modify data

Example:

**class** Student:

```
def __init__(self, marks):  
    self.__marks = marks
```

```
def get_marks(self):  
    return self.__marks
```

```
def set_marks(self, marks):  
    if 0 <= marks <= 100:  
        self.__marks = marks  
    else:  
        print("Invalid marks")
```

Usage:

```
s = Student(85)
```

```
print(s.get_marks())
```

```
s.set_marks(95)
```

But:

```
s.set_marks(150)
```

will be rejected.

The class controls the state.

---

## 11. Why Is This Better?

Without encapsulation:

```
s.marks = -500
```

With controlled modification:

```
s.set_marks(-500)
```

The method can validate it:

```
if 0 <= marks <= 100:  
    self.__marks = marks
```

Therefore:

Outside world

```
|  
↓  
set_marks(95)  
|  
↓  
validation  
|  
↓  
__marks = 95
```

The object's internal state isn't blindly modified.

---

## 12. Pythonic Encapsulation: @property

Python commonly uses **properties** instead of Java-style getters and setters.

Example:

```
class Student:

    def __init__(self, marks):
        self.__marks = marks

    @property
    def marks(self):
        return self.__marks

    @marks.setter
    def marks(self, value):
        if 0 <= value <= 100:
            self.__marks = value
        else:
            print("Invalid marks")
```

Now:

```
s = Student(80)
```

```
print(s.marks)
```

Python internally calls:

```
s.marks
  ↓
marks()
```

And:

```
s.marks = 95
```

calls the setter.

So you get a clean interface:

```
s.marks = 95
```

while the implementation remains controlled.

---

## 13. Real-World Example

Think about a car.

You interact with:

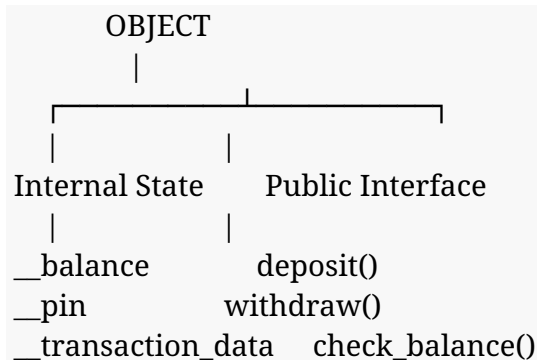
```
car.start()
car.accelerate()
car.brake()
```

You don't need to directly manipulate:

```
fuel_injection
engine_temperature
transmission_state
rpm
air_fuel_ratio
```

The car provides a controlled interface.

Similarly, in OOP:



The user interacts primarily with the interface.

The internal implementation can change without changing how the user uses the object.

That's a major benefit of encapsulation.

---

## 14. Encapsulation vs Data Hiding

These are related but **not identical**.

### Encapsulation

Bundling data and behavior together and controlling access to the object's internal state.

## Data hiding

Restricting or discouraging direct access to internal implementation details.

Example:

```
class BankAccount:
```

```
    def __init__(self):  
        self.__balance = 10000  
  
    def deposit(self, amount):  
        self.__balance += amount
```

Here:

\_\_balance

↓

Data hiding

BankAccount + \_\_balance + deposit()

↓

Encapsulation

---

## 15. Encapsulation vs Abstraction

These two are often confused.

### Encapsulation

Focuses on:

**How do we package and control the object's data/state?**

### Abstraction

Focuses on:

**What details should the user need to know?**

Example:

```
car.start()
```

You don't need to know the internal engine process.

That's **abstraction**.

Meanwhile, keeping:

```
engine_state
fuel_system
ignition_state
```

inside the object and controlling their modification relates to **encapsulation**.

A useful distinction:

```
Encapsulation → Protect/control the implementation
Abstraction  → Hide unnecessary complexity
```

They often work together.

---

## 16. Encapsulation vs Inheritance

These are completely different OOP concepts.

### Encapsulation

```
Class
├── data
└── methods
  ↓
control access
```

### Inheritance

```
Parent
  ↓
Child
  ↓
reuse/extend behavior
```

For example:

```
class Animal:
    def eat(self):
        print("Eating")
```

```
class Dog(Animal):  
    def bark(self):  
        print("Barking")
```

That's inheritance, not encapsulation.

---

## 17. Complete Example

Here's a good encapsulation example:

```
class BankAccount:  
  
    def __init__(self, owner, balance):  
        self.owner = owner  
        self.__balance = balance  
  
    def deposit(self, amount):  
        if amount > 0:  
            self.__balance += amount  
        else:  
            print("Invalid deposit")  
  
    def withdraw(self, amount):  
        if amount <= 0:  
            print("Invalid withdrawal")  
  
        elif amount > self.__balance:  
            print("Insufficient balance")  
  
        else:  
            self.__balance -= amount  
  
    def get_balance(self):  
        return self.__balance
```

Usage:

```
account = BankAccount("Abhi", 10000)  
  
account.deposit(2000)  
account.withdraw(3000)
```

```
print(account.get_balance())
```

Output:

```
9000
```

Notice what the outside code **doesn't do**:

```
account.__balance = -50000
```

Instead, it uses:

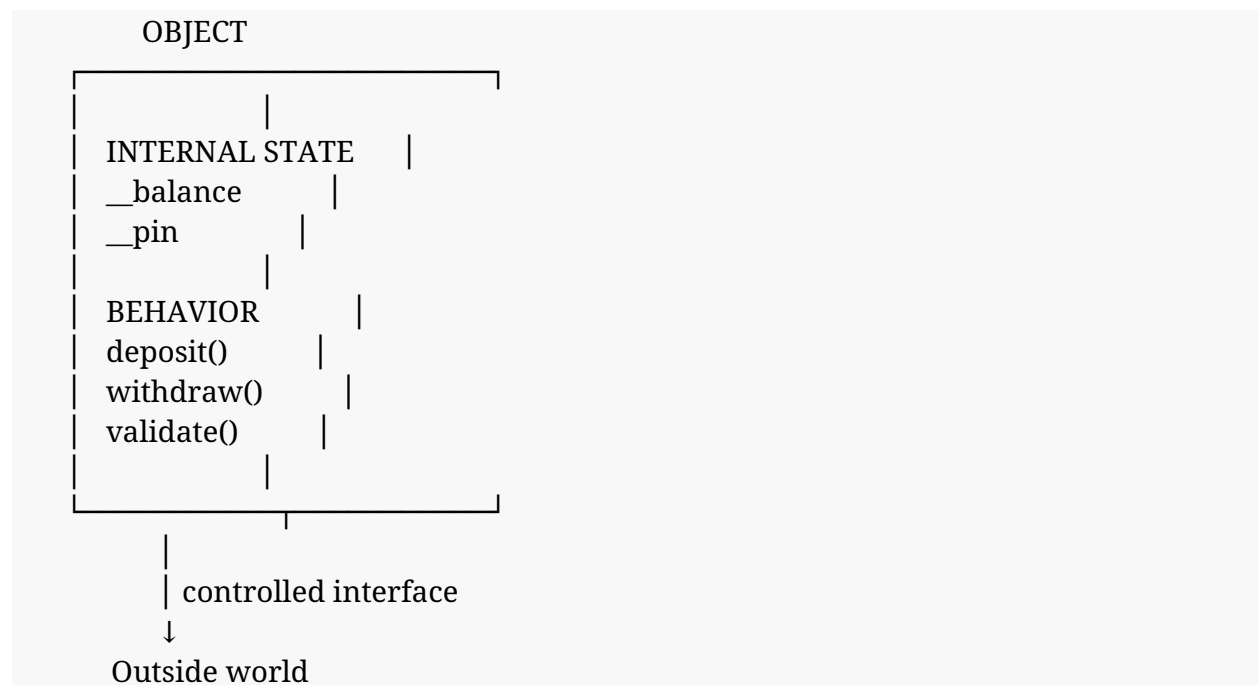
```
account.deposit(...)
account.withdraw(...)
account.get_balance()
```

The class controls its own state.

---

## 18. The Mental Model You Should Remember

Think of an object as a **capsule**:



The outside world shouldn't need to know or directly manipulate everything inside.

---

## 19. Exam/Interview Definition

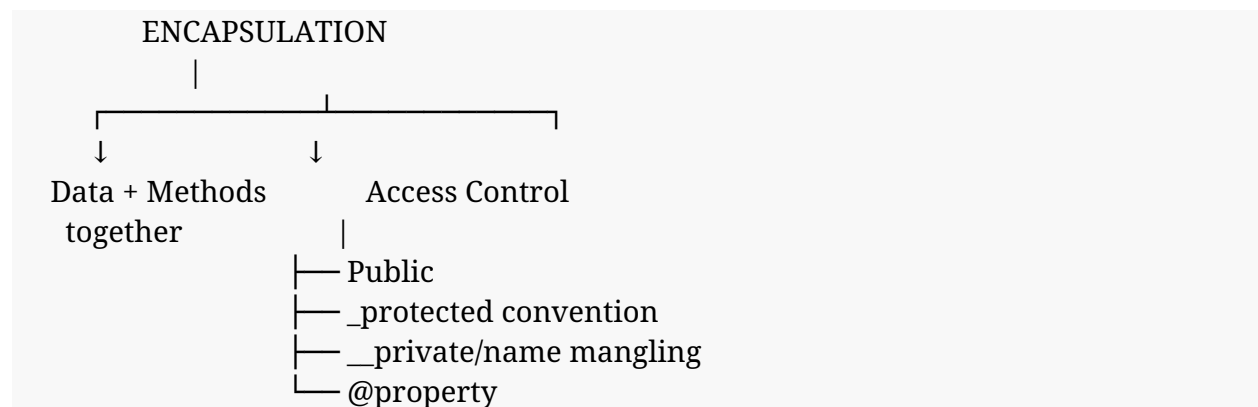
If someone asks:

**“What is encapsulation in Python?”**

A strong answer is:

**Encapsulation is an OOP principle in which data and the methods that operate on that data are bundled together inside a class, while access to the object’s internal state is controlled through a defined interface. Python supports encapsulation using public members, naming conventions such as `_name`, name mangling with `__name`, and properties.**

And remember the hierarchy:



**The key idea isn’t “make variables private.”**

The deeper idea is:

**An object should manage and protect its own state through controlled operations.**